

Towards Effective Productivity Measurement in Software Projects

Gibeon Soares de Aquino Júnior
Federal University of Pernambuco and
C.E.S.A.R. - Recife Center for Advanced
Studies and Systems
Recife, Brazil
Email: gibeon.aquino@cesar.org.br

Silvio Romero de Lemos Meira
Federal University of Pernambuco and
C.E.S.A.R. - Recife Center for Advanced
Studies and Systems
Recife, Brazil
Email: silvio@cesar.org.br

Abstract—Measuring the productivity in software development projects is a historical complex problem. The main problem resides in quantifying the relevant results produced in a software project. Different organizations and persons have distinguished notion about the concept of what is produced and its associated value. Therefore, the appropriate model for measuring productivity in software organizations needs to be unique to each type of organization. In addition, we encountered, based on a broader investigation about the productivity metrics and measurement studies published in literature, that there seems to be a trend towards the use of what we called Value Size based Metrics for productivity measurement. Based on these findings, we defined a process to support the definition of productivity measurement models in software organizations and which takes into account the difficulties of measuring the productivity and reflects the best practices in measurement programs. The adoption of a productivity measurement model based on value helps the organization to focus its efforts to improve productivity in the direction suitable with the high management vision.

Keywords—Productivity; Measurements; Software process;

I. INTRODUCTION

The global environment requires that companies increasingly improve their levels of quality and reduce their production costs. The main way to reduce production costs, specifically in software development, is through increasing of productivity; in other words, develop further with a lower cost or effort. For this reason, the search for productivity improvement has been a constant concern in organizations. However, to improve productivity it is necessary, first, to understand how to measure it. DeMarco [20] emphasizes that the first step towards improve and control is to measure, as his statement says: “You can’t control what you can’t measure”. Therefore, this article in particular, will explore the question: **How to measure productivity?**

Productivity is conceptually a simple measure, and in general can be defined as the ratio between what is produced and what is consumed: $productivity = output/input$. The concept of productivity is widely used in many expertise areas, from agriculture to economics. Despite the idea of productivity being very simple and popular in other areas, specifically in software development it is a historically complex problem. According to Jones [32] the lack of

precise and non-ambiguous definition of productivity metric has been a source of problems in software projects. This difficulty in measuring the productivity in software development is not caused by the productivity itself, but by the intrinsic characteristic that is the meaning of *output* in software projects context. Quantify the results produced in a software project in terms of size, complexity or value to the customer is a very complex problem [15].

In addition, Scacchi [45], reports on its study of productivity metrics that it is evident that the studies on this matter is fundamentally inadequate and potentially flawed. Moreover, depending on how and which indicators are measured the conclusions about the productivity can be completely different.

The extensive literature on productivity improvement and measurement confirms the scientific interest on this subject [3], [39], [45]. Many issues are still without answers and there are still many myths about the problem of productivity. This article investigates several studies published in literature on metrics and measurement of productivity in software development projects in order to raise the main lessons learned on this matter. In particular, it proposes the adoption of a process to support the definition of models for measuring productivity based on the understanding that each organization must define its own way to assess their productivity.

II. PRODUCTIVITY MEASUREMENT: A SURVEY

Measuring productivity of projects has been a major challenge for practitioners and researches in software development. Despite the difficulty of effectively measuring the software projects productivity, much effort has been performed in attempt to obtain ways to assess productivity, following the philosophy of Gilb’s law [27] who state that “*Anything you need to quantify can be measured in some way that is superior to not measuring it at all*”.

The first empirical studies have emerged in the mid-60 in the System Development Corporation and were based on statistical analysis of data from the software development projects in U.S. army [23], [24]. These studies were intended to identify factors affecting the cost of software projects,

based on productivity measurements, and thus improve the estimation process.

Several aspects of the problem about productivity metrics in software projects are still open, showing the importance of exploring this area, from both a scientific and practical viewpoint. However, before establishing future directions for this area of research it is essential to exploit the results of studies from the past. Due to the large numbers of studies, using different metrics of productivity, it is essential to group them with the aim of effectively investigate them. The studies reviewed in this section shall be grouped according to the approach of *output* measurement and will be described in more details in the following sections, as the following categories: **Physical Size**, **Design Size**, **Functional Size** and **Value Size**.

A. Physical Size Based Metrics

This section considers the studies that used SLOC, variations or derivations thereof. Examples of these are variations of SLOC are *Delivery Source Instructions*, *Executable Line Of Code* and *Developed Statements*. The derivations of SLOC metrics are calculated from the source code, such as *Non-Commented Source Code* and *Effective Source Lines of Code*.

Although SLOC being something quite basic it is widely used in studies of productivity as the output measure in the evaluation of productivity. In studies that follow we identify that different metrics are used for output, but all based on the counting of physical lines of code. Different names are used to represent the same concept, such as *Delivery Source Line*, *Machine Instructions*, *Delivery Source Instruction* and *Executable Line Of Code*. Another difficulty encountered when compiling these studies was that on most of these, there is no precise specification of what is considered in the counting of these metrics (e.g., comments, blank lines, statements, etc.). And because this, further analysis and comparison between them becomes more difficult.

Initial studies about this matter [25], [34], [43] used a metric conceptually equal to SLOC: *Man-Months per Machine Instructions*. Aron [5], in his studies on projects at IBM, widely used a metric of SLOC. Subsequently, Walston and Felix [51] conducted a broader study in the IBM Federal Systems Division and used *DSL/MM* as the *productivity* metric, where *DSL*(Delivery Line Source) means that only those lines of code delivered as part of the product.

At the end of the '70s, Jones [32] states that productivity and metrics quality, in terms of lines of code are paradoxical, since LOC per unit of effort tends to emphasize more the size and format of the programs than the efficiency and quality of these. According to him, this metric tends to penalize high-level languages for languages such as Assembly. For this reason the LOC is not a good indicator of economic productivity, despite being frequently used. Boehm [15], for

example, showed that a well formatted program can be three times its original size.

Even recognizing the shortcomings of SLOC based metrics, in the 80s the scientific community, as was the case of Lawrence [35], Bailey and Basili [7], Boehm [13], [14] and Vosburgh et al. [50] continued to use it in their studies of productivity and estimation. Grady and Caswell [28] and Card et al. [17] used the metric of *NCSS*, based on SLOC, to evaluate the productivity of projects in HP and Software Engineering Laboratory (SEL), respectively. Recently, studies on productivity also used SLOC metric, such as MacCormack [37], Refear [44] and Moses et al. [42].

B. Functional Size Based Metrics

This section considers the studies which use *output* measures based on number of features perceived by the user. The most common examples of these types of metrics that are *Function Points* [1] and *Use-case Points*.

Behrens [10] widely used metrics based on function points to measure the projects productivity. Banker [8], Parkan et al. [16], Jones [31] and Moses et al. [42] also used function points as *output* measure. On the other hand Symons [47] and Maxwell and Forselius [38] criticizes the use of function points and suggest adjustments to the metric.

Even Arnold and Pedross [4] used the unit of use case points as a measure of output, which is a unit of measurement based on function points, but based in a different concept to account it.

Function Point became one of the most known and used metric to quantify the size of software and consequently the productivity. Although this, it is very criticized and questioned because several assumptions and definitions do not have a theoretical apparatus that validate it [22], [26], [31], [48].

C. Design Based Metrics

At this group, studies that uses metrics related to the design are present, they use metrics as number of modules, classes or related things. These metrics have a correlation with the SLOC, but are based on different aspects.

Chatman [49], for example, suggests using a new metric, called *Change-Point*. Moser and Nierstrasz [41] used a new metric called System Meter. The System Meter is an object oriented metric and is calculated as the sum of the sizes of all objects in the system. Morasca and Russo [40] used three different metrics to assess the size of productivity, they were: Function Points, LOC and Number of Modules. The conclusion of the authors was that the first is useful for the outside perspective (Customer) and the latter two for the internal (Team and Organization).

D. Value Based Metrics

Studies at this group use a more modern vision to assess the productivity and use *output* metrics based in added

value, or use multidimensional models that assess different aspects of what is produced in a software project.

Duncan [21] criticizes the use of SLOC based metrics as a productivity measure. He says the key process of software development is the transformation of ideas into products. So to measure the real productivity of software development, we need to measure how effectively and efficiently it can transform ideas into software. For Anselmo and Ledgard [2], measure the amount of features of software, using traditional techniques such as Function Point and Use case point is insufficient, since such methods assess so simple the level of functionality complexity.

According to Hastings and Sajeev [29] the metric of function point does not adequately address the internal complexity of the system, which can result in a disproportionate measure for each type of software. Based on this they proposed a new measure, known as *Vector Size Measure (VSM)*, which considers both the complexity and functionality to calculate the size of software, i.e. the unit of output.

Stensrud and Myrtveit [46] criticize the traditional productivity measurement model based on the equation $produtividade = output/input$, because according to them a project of software has various measures *output* and therefore a multidimensional model would be more appropriate to effectively evaluate the project productivity. In this work they propose to use a metric called *Data Envelopment Analysis Variable Returns to Scale (DEA VRS)*, which considers multiple variables of *input* and *output*.

Following the same idea of using a multidimensional approach to assess productivity, Abran and Bunglione [36] define a three-dimensional model to evaluate the software projects performance, called QEST (**Q**uality factor + **E**conomic, **S**ocial and **T**echnical dimensions). In this study they use geometric concepts to define how to evaluate the performance, where the dimensions are represented by the factors **E**, **S** and **T** and adjusted by a factor **Q** based on qualitative perceptions.

E. Concluding Remarks about Productivity Measurement

In fact, there is no direct and safe response for the question: **What is the best software productivity metric?** The simplest and most commonly used metric is the SLOC and its derivations, but these are admittedly problematic for this purpose. Moreover, value-based metrics, which involve assessment of productivity in many dimensions, show a good alignment with the idea of producing value and are gaining popularity today, but are more complex, less common and are still maturing.

An ideal metric of productivity should be able to measure how effectively and efficiently to turn ideas into software products [21]. Also, according to Boehm [11], [12] quantifying the outcome of which is produced in a software project should not be reduced to number and complexity

of requirements or code, but to the value of the product to the stakeholders. In this thought, the productivity metrics based on value, which are based on multiple dimensions, and which correspond to what the stakeholders consider as the value being produced, is the most complete and reliable to the organizational core business perspective.

Morasca and Russo [40], for example, concluded in a study done using both Function Point and SLOC as a measure of output, which both are interesting depending on the perspective of analysis used. According them, different stakeholders perceive the result of the work differently and evaluate the *outcomes* of a project differently. This point reinforces the idea that to make use of a good metric of productivity is essential (1) understanding the different *outcomes*; (2) to understanding the importance of each of these to the stakeholders; (3) knowing the perspective which the productivity is being evaluated. The item (3), in particular, is extremely important because the same project can be extremely productive from a technical viewpoint (e.g., producing many of Function Points or SLOC per hour), but a disaster in the viewpoint of the business (if you cannot have a good ratio between revenue and expenditure).

Many divergences exist over what is produced during the project and what should be measured. Even some intangibles aspects such as quality and value of what is being produced as a result of a project, become it difficult to define exactly “What is productivity in software projects”. In this context, some argue that to measure, in some way, is better than simply not measure at all [27], [30]. However, Austin [6], alerts about the risks of measuring dimensions that are not strongly linked to the phenomenon being observed. He says there is a strong possibility that by doing this the performance of the process or phenomenon being measured will worsen, despite the contrary opinion of the outcome metrics. This phenomenon was widely observed by the author in different types of organizations, receiving the name of **Dysfunction**. In particular, productivity measurement initiatives presents a great availability to emergence and deployment of dysfunction. For this reason, is recommended to be careful while setting a model to assess the projects productivity in software organizations.

More effort needs to be done towards the definition and use of multidimensional models for measuring productivity. There are several evidences of the problems arising from the use of traditional models that consider only a single metric, or more specifically to assess a single dimension of the production of software [6], however many reports and conclusions are done based on the use of these models. To change the state of the art in software productivity, the first step is to develop specifically expertise in measuring productivity. For this, the use and development of more modern approaches, which seek to achieve the real dimensions of value produced in software projects are needed.

The realization of the dimensions of value in software

development is not a easy task, several of them are difficult and costly to monitor or are simply intangible. In addition, each organization has a singular work way and different viewpoint about the value of *outputs* produced as a result of a software project. Even this viewpoint varies within the organization, depending on the level of those involved in the evaluation of productivity measures. Therefore, there is no way to define a universal model for measuring productivity to any type of software organization and projects. Each type of organization must define its own model for productivity measurement, which can even vary depending on the types of projects that runs it.

III. A EFFECTIVE PRODUCTIVITY MEASUREMENT PROCESS

Different organizations have distinguished notion about the concept of produced value. The outputs of a software development process are evaluated differently, depending on the type and purpose of the organization, the persons involved or even the type of project where the measurement is applied. Therefore, the appropriate model for measuring productivity in software organizations is unique to each type of organization. In this context, a well defined process, which takes into account the difficulties of measuring the productivity and reflects the best practices in measurement programs, can help the organization to maximize the chances of success in defining and adopting a model of effective productivity measurement.

Based on the approach of Hubbard [30], which argues that anything can be measured, on the recommendations and care cited by Austin [6] and on productivity measurement literature review, this paper proposes a process to support the definition of productivity measures in software projects. The result of applying this process produces a model of productivity measurement that takes into account the most relevant dimensions of value to the organization, based on the vision of those interested in monitoring and evaluating the projects productivity.

A. The Process Model

The proposed process model consists of 6 activities, as can be seen in Figure 1, they are: Establish the context, Specify the perspective(s), Identify dimensions of value, Define the model for productivity measurement, Refine the measurement model and Planning the adoption of the model. This process can be executed several times, as the organization is maturing its ability of productivity measurement. In other words, it means that the organization don't need to define the type of measurement completely at the first time. In each cycle of the process execution, the scope of operation can increase and the productivity measurement model be evolved.

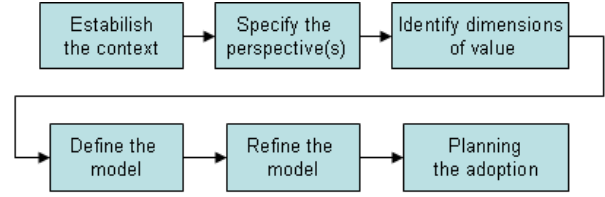


Figure 1. Productivity Measurement Process.

1) *Establish the context*: The scope in which the measurements will be applied and the stakeholders involved strongly influence how the process of defining productivity measurement will work. Furthermore, the purpose of measurement, or more specifically, how the results of the measurements will be used by the organization, defines the strategy to be adopted both for the definition and for the deployment of the model for productivity measurement. In this activity the following steps must be executed in order to accomplish the startup purpose: **Define the scope**, **Define the purpose**, **Identify the stakeholders** and **Establish commitments**.

Projects or areas of the organization whose projects will be measured from the productivity viewpoint must be defined and known by all. The step **Define the scope** helps to identify the complexity of the definition and implementation of the productivity measurement initiative. **Define the purpose** is another important step inside this activity. The purpose behind the measurement program must be defined and known by all. Common examples are the use measurement results for: assessment of individuals or teams, rewards, information and competitiveness improvement. To identify the most relevant stakeholders to the definition of the productivity measurement model (**Identify the stakeholders**) is important to look for those who need to make decisions based on results of the productivity analysis of the software development projects in the organization. With the scope defined, the purpose established and the stakeholders identified, we need to define responsibilities within the definition of the measurement model, gain the commitment of all involved, and in particular, the organization top management. The step **Establish commitments** has this goal.

2) *Specify the perspective(s)*: Different stakeholders perceive and evaluate the inputs and outputs of a production process differently, depending on their position within the organization. A project manager, for example, can evaluate the productivity of a project depending on the quantities of features generated from the effort consumed, as a CFO (*Chief Financial Office*) can understand the productivity as the amount of funds generated from the amount of resources consumed. Other actors may want to add other dimensions, beyond the traditional. For example, the CTO (*Chief Technology Office*) may want to take the degree of technology innovation involved in the project, the area of

human resources, the team satisfaction with the project, the SQE (*Software Quality Engineer*), the quality of product.

A production process in an organization can be seen as a function that consumes inputs and produces outputs, resulting from the process implementation, according to the Figure 2. In this case, the productivity should be calculated using the Equation 1, where I_j represents the quantity of the input j , O_i the quantity of the output i , Vi_j , the unit value of the input j and Vo_i , the unit value of the output i .

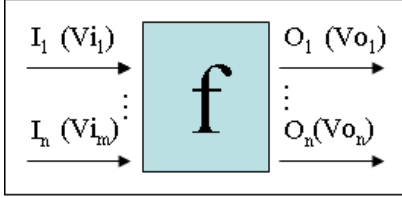


Figure 2. Simplified production process.

$$Productivity = \frac{\sum_{i=0}^n (O_i \cdot Vo_i)}{\sum_{j=0}^m (I_j \cdot Vi_j)} \quad (1)$$

The model in Figure 2 reflects a simplification of reality. Indeed, depending on the context (more specifically the observer) the unit value assigned to Vi_j and Vo_i may vary, and as consequence, the result of the productivity measurement to be relative to the context under which it is evaluated. The Figure 3 reflects a more realistic vision of the production process and evaluation of productivity in organizations. In this model there are several perspectives and inside each of them the unit values of inputs and outputs vary and may even be zero in some perspective (meaning to say that in that perspective that output or input is irrelevant for the productivity measurement, according to the vision of the person concerned in the measurement). The Equation 2 incorporates this change in the productivity calculus, where the $Productivity_x$, represents the productivity in perspective x , Vo_{xi} and Vi_{xj} , represents the unit values of the output i and input j , respectively, inside the perspective x .

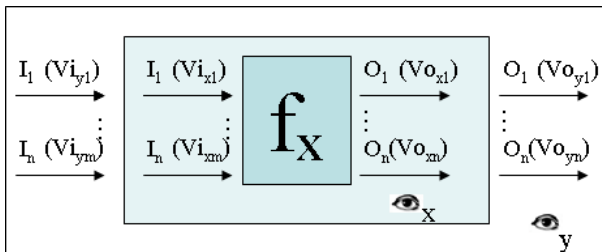


Figure 3. Production process with multiples perspectives.

$$Productivity_x = \frac{\sum_{i=0}^n (O_i \cdot Vo_{xi})}{\sum_{j=0}^m (I_j \cdot Vi_{xj})} \quad (2)$$

As productivity is relative and depends on the vision of the person inside it, we can say that **the productivity is relative**, because it depends on the perspective where it is defined and evaluated. Therefore, the first step in definition of metrics of productivity in an organization is to understand what are the most important perspectives and who are involved in each of them. The following steps should be executed to achieve this aim: **Identify the perspectives, Prioritize and merge the perspectives and Describe the perspectives.**

To **Identify the perspectives**, it is necessary to group the stakeholders in different perspectives according to their interests and perception of the meaning of productivity. A source for initial identification of these perspectives are the hierarchical levels and decision makers areas within the organization. It is essential that some of the perspectives raised are related to strategic viewpoint (or the main level/area of decision makers), however it can be very high level and not be directly related to operational adjustments that are usually done and felt as results of processes improvement initiatives. Therefore, it is necessary to consider, also, an operational perspective, closer to the organization core production area, represented in Figure 3.

The next step is **Prioritize and merge the perspectives**. The excess of perspective may hinder the productivity measurement model definition. For this reason it is suggested to use an incremental approach, where initially, only one or at most two perspectives are used, and as the model is being used and matured, new perspectives may be considered. At this moment the suggestion is to prioritize the most important perspectives and combine them if possible to minimize their quantity.

Finally, it is important to **Describe the perspectives**, whose aims is avoiding poor understanding of the perspectives used in the model, it is suggested to specify them in terms of the purpose, participants, stakeholders and related levels/areas.

3) *Identify dimension(s) of value:* As already seen in Section II-E, the metrics of productivity based in value and multiple dimensions are considered most suitable for productivity measurement software development projects. For this reason, an activity that should be done at this moment, after having defined the perspectives that will be part of the scope of the measurement model is the identification of elements (inputs consumed and outputs produced) relevant to the value concept viewpoint. In a software development project, there are a variety of outputs produced, which include from tangible aspects such as features and suitability to customer needs, to intangible aspects such as quality, image and experience. Already on the inputs consumed, the most commonly used are effort or financial resources. Therefore, greater attention should be given to identifying the outputs of the software production process. As part of this activity, we detail them in the following steps: **Identify the relevant inputs, Identify the relevant outputs and**

Assess the degree of importance of the outputs.

For each specified perspective, it is necessary to **Identify the relevant inputs**, according to the stakeholders vision. This step involves assessing whether the most appropriate input to measure is the effort, financial resources or some other specifics of this perspective. The next step to be executed is to **Identify the relevant outputs**, where for each perspective specified it is necessary to identify which outputs of the production process are most important, according to the stakeholders view inside the perspective. These outputs can be more easily identified from the analysis of the following dimensions: **Product** – Produced outputs that add value to the product, such as number and complexity of features, performance and usability; **Client** – Produced outputs that add value to the customer, such as satisfaction and financial return; **Organization** – Produced outputs that add value to the organization that developed the product, such as financial return, image, knowledge and patents; and, Finally, **Team** – Produced outputs that add value to the development team, such as satisfaction and experience.

The last step in this activity is to **Assess the degree of importance of the outputs**. Not all outputs can be monitored, either because they are intangible, or because the costs of measurement are too high. Therefore, it is necessary to set levels of importance, based on the stakeholders vision, according with the following scores: **3** – Output that is essential, without it the model is in the risk of not reflecting the true idea of productivity; **2** – Output is important, but her evaluation may be left out at a first moment; **1** – Output would be interesting, though without it the model for assessing productivity approximates to the ideal.

4) *Define the model for productivity measurement:* After the inputs and outputs has been identified it is necessary to define how the productivity will be calculated from these, for each perspective. In particular it will be necessary to define the structure of Equation 2, i.e, to define exactly who are the n metrics O_i and m metrics I_j . For this, the following steps must be performed several times, until the definition of the model for productivity measurement is reached: **Define the candidates metrics**, **Evaluate the measurement costs**, **Prioritize metrics** and **Assign the unit value for the inputs and outputs**.

To define the measurement model the first step is to **Define the candidates metrics**. In this step, you must define the metrics that can be used to quantify the outputs identified in previous activity. In some cases, where there is greater difficulty in defining a metric directly related to the output, you can, as an alternative, define a metric that is indirectly related to the effect that you want to quantify. One popular approach that could be used to help the definition of these metrics is the GQM (Goal-Question-Metric) approach [9], which define a method of refining an abstract concept, as goal, into a metric. Using the same idea, it could be used to refine the output concept into metrics. Others approaches [19],

[33], with the same aim, exist and could be used as tools to define and validate the metrics related with the relevant outputs in the productivity measurement model.

Next, it is necessary to **Evaluate the measurement costs**, because although it is easy to specify the metrics, some of them to be viable requires a great effort, which may involve changes to the process, culture and tools used in the organization. For this reason, it is important to evaluate how prepared is the organization to collect each candidate metric. For this, you must assign values to each metric according to the assessment of cost of adoption and collect it inside the organization, according to following scores: **5** – Currently, the metric is collected and no additional effort is required for its adoption; **4** – The metric is not collected directly, but the information necessary for its accounting is already available in an easy; **3** – The metric is not currently collected and the information necessary for their accounts are not available, but it is estimated that it is not difficult to obtain them; **2** – The metric is not currently collected, the information necessary for their accounts are not available and changes in process, culture or tools are required to obtain it, but it is believed that it can be collected objectively; **1** – The metric is not currently collected, the information necessary to account it are not available, changes in process, culture or tools are necessary to obtain it and there is some subjectivity in its calculation.

Based in the results of previous steps, now it is necessary to **Prioritize metrics**. Metrics whose collection is expensive and that are not directly linked to relevant outputs, according to the stakeholders views, should be abandoned. To support this analysis, you should calculate what we called **Metric Fitness**. This can be obtained by the product of the cost of evaluation with assessment of the importance of output associated with it. Metrics with high fitness should be part of the first versions of the measurement model, while metrics with low value should be reconsidered.

To close the cycle of defining productivity measurement model, or more specifically, the instantiation of the Equation 2 the values of Vo_{x_i} e Vi_{x_j} should be defined. This definition must be based on the stakeholders vision about the value of one unit of each of the inputs and outputs at a particular perspective. A technique that can be used to define such values is the MACBETH (Measuring Attractiveness by a Categorical Based Evaluation Technique) [18], but another specific techniques based in specialist know-how can be used too.

5) *Refine the measurement model:* At this moment a preliminary version of the model already exists, but there are some necessities adjustments to be done to improve the chances os sucess, these adjustments can be defined as the following steps: **Evaluate Risks**, **Validate the model** and **Adjust the model**.

The first step in this activity is to **Evaluate Risks**. To prevent the dysfunction of the defined measurement model

it is important to evaluate their weak points and those which offer risks to the successful implementation of the model. Among the main sources of problems, we can highlight the following: metrics that do not have a bidirectional match with output which it is proposed to quantify (if the metric can have its value affected regardless of the output that it is related), metrics whose collection can be easily corrupted and metrics whose understanding and specification is not very clear at all. A risk assessment may indicate that the model is appropriate to the context of the organization and create subsidies to decide which adjustments must be done. Also, it should define strategies for handling risks which are useful for planning the deployment of the model.

Validate the model is the next step to refine the model. With the proposal of the measurement model finished, the main stakeholders in the process should validate it. This validation can be a simple presentation, experiments on data from past projects, the implementation of pilot projects, etc.

Finally it is necessary to **Adjust the model**. Based in risk assessment of the proposed model and the validation performed, adjustments in the model should be done to minimize risk and turn it more aligned with the stakeholders perceptions.

6) *Planning the model adoption*: The deployment of the measurement model across the organization is still a project, as well as any other that has a scope, cost, time, risk. Therefore, it is necessary to plan the adoption, in order to define a strategy to enhance the chances of success in its adoption. The deployment is most complex part of the measurement process, for this reason a careful and labored planning is suggested.

B. Process Adoption Issues

The first step that must be done even before the start of the measurement model definition, is to direct efforts to persuade opinion formers and decision makers about the risks and shortcomings of traditional productivity measurement models. Such models, assess a single dimension of value, which in software development, where produced outputs are more than SLOC and functional size, become not appropriate to measure the productivity in terms of produced value. The understanding of these deficiencies, the agreement about the benefits of a productivity measurement model that considers multiple dimensions of value and the alignment with the senior's management vision about what is most relevant from the viewpoint of production are essential requirements to start the execution of this proposed process.

Moreover, it is important to show to everyone that this type of measurement model is based on the premise that each organization is unique in relation to the vision of assessment produced outputs value by his production process and for this reason has no sense to compare different organizations. It is not the purpose of this type of model to be used as tool for external benchmarking. Its main aim is to serve

as mechanism for internal benchmarking, between similar projects within the organization, and in this way help the organization to identify existing practices that increase the produced value in direction expected by senior management. Baley and Basili [7], as we, do not agree with metrics for comparison of productivity between organizations. They claim that measures of productivity should be specific to each organization.

Finally, it is recommended to show that this definition may be based on an incremental approach, which initially sets up a simple model, requiring low investment in its definition and adoption. Later, as the organization matures its ability to measure the productivity and begins to realize the benefits of this type of model, the process of definition can be rerun considering a broader and more complex scope.

IV. CONCLUSIONS AND FUTURE WORKS

The adoption of a productivity measurement model based on value helps the organization to focus its efforts to improve productivity in the direction consistent with the high management vision. However, realizing abstract idea of value in a measurement model is not an easy task. Different organizations have distinguished notion about the concept of produced value. Therefore, the appropriate model for measuring productivity in software organizations needs to be unique to each type of organization. For this reason, the process proposed here is extremely important to guide the organization to define a suitable productivity measurement model, which takes into account the difficulties of measuring the productivity while reflects the best practices in measurement programs.

The model described here is a proposal that still needs to mature and detailed, although it take into account a broad literature review on this matter. Currently, it is being used in C.E.S.A.R. - Recife Center for Advanced Studies and Systems, a medium size Brazilian software development organization, and is part of a doctoral research being conducted in the Federal University of Pernambuco in Brazil too. Some parcial results already be obtained with his adoption, but more time is necessary to observes the benefits of this kind of approach.

Clearly, knowledge about measurement of productivity in software projects need to evolve, much effort still needs to be done in order to mature this practice. The definition of this process is a step in this direction, though other aspects which are not part of the scope of definition of a model for productivity measurement, need to be explored in order to have a consistent growth of this area. One of these problems, which need to be addressed is the definition of a process, which compile the best practices of metrics programs and the specific problems of productivity measurement, to support the development of productivity measurement models based on value. Another relevant aspect to be further explored is how to use the productivity measurement model in each type

of organization. In other words is defining the best way, particular to one type of organization, of using the model, based on the results of measurements, enabling productivity improvement, while avoiding the typical problems of these types of initiatives, known as dysfunction. In particular, a kind of use which can best be investigated is the use for motivation, i.e. to serve as basis for reward system.

REFERENCES

- [1] Alan Albrecht. Measuring application development productivity. In I. B. M. Press, editor, *IBM Application Development Symp.*, pages 83–92, October 1979.
- [2] Donald Anselmo and Henry Ledgard. Measuring productivity in the software industry. *Commun. ACM*, 46(11):121–125, 2003.
- [3] Gibeon Aquino and Silvio Meira. Software productivity measurement: Past analysis and future trends. In *SETP'09: Proceedings of Third International Conference on Software Engineering Theory and Practice*, Orlando, Florida, USA, July 2009. ISRST.
- [4] M. Arnold and P. Pedross. Software size measurement and productivity rating in a large-scale software development department. *Software Engineering, 1998. Proceedings of the 1998 International Conference on*, pages 490–493, Apr 1998.
- [5] J. D. Aron. Estimating resources for large systems. In *In NATO Conference Report on Software Engineering Techniques*, pages 68–79, Rome, Italy, October 1969. Brussels: NATO Science Committee.
- [6] Robert Austin. *Measuring and Managing Performance in Organizations*. Dorset House Publishing Company, 1996.
- [7] John W. Bailey and Victor R. Basili. A meta-model for software development resource expenditures. In *ICSE '81: Proceedings of the 5th international conference on Software engineering*, pages 107–116, Piscataway, NJ, USA, 1981. IEEE Press.
- [8] Rajiv D. Banker, Srikant M. Datar, and Chris F. Kemerer. A model to evaluate variables impacting the productivity of software maintenance projects. *Manage. Sci.*, 37(1):1–18, 1991.
- [9] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. 1994.
- [10] C. A. Behrens. Measuring the productivity of computer systems development activities with function points. *IEEE Trans. Softw. Eng.*, 9(6):648–652, 1983.
- [11] Barry Boehm. Value-based software engineering: reinventing. *SIGSOFT Softw. Eng. Notes*, 28(2):3, 2003.
- [12] Barry Boehm and Li Guo Huang. Value-based software engineering: A case study. *Computer*, 36(3):33–41, 2003.
- [13] Barry W. Boehm. *Software Engineering Economics*. Prentice Hall PTR, Upper Saddle River, NJ, USA, 1981.
- [14] Barry W. Boehm, James F. Elwell, Arthur B. Pyster, E. Donald Stuckle, and Robert D. Williams. The TRW software productivity system. In *ICSE '82: Proceedings of the 6th international conference on Software engineering*, pages 148–156, 1982.
- [15] B.W. Boehm. Improving software productivity. *Computer*, 20(9):43–57, September 1987.
- [16] Parkan C., Lam K., and Hang G. Operational competitiveness analysis on software development. *Journal of the Operational Research Society*, 48(9):892–905, September 1997.
- [17] D. N. Card, F. E. McGarry, and G. T. Page. Evaluating software engineering technologies. *IEEE Trans. Softw. Eng.*, 13(7):845–851, 1987.
- [18] Jean-Claude Vansnick Carlos A. Bana e Costa. Applications of the macbeth approach in the framework of an additive aggregation model. *Journal of Multi-Criteria Decision Analysis*, 6(2):107–114, 1997.
- [19] Joseph P. Cavano and James A. McCall. A framework for the measurement of software quality. pages 133–139, 1978.
- [20] Tom DeMarco. *Controlling software projects : management, measurement and estimation*. Yourdon Press, 1982.
- [21] A. S. Duncan. Software development productivity tools and metrics. In *ICSE '88: Proceedings of the 10th international conference on Software engineering*, pages 41–48, Los Alamitos, CA, USA, 1988. IEEE Computer Society Press.
- [22] Lem O. Ejiogu. *Software engineering with formal metrics*. QED Information Sciences, Inc., Wellesley, MA, USA, 1991.
- [23] L. Farr and B. Nanus. Factors that affect the cost of computer programming. Technical Report TM-1447/000/02, System Development Corporation, Santa Monica, California, July 1964.
- [24] L. Farr and H. J. Zagorski. Factors that affect the cost of computer programming: A quantitative analysis. Technical Report TM-1447/001/00, System Development Corporation, Santa Monica, California, August 1964.
- [25] Leonard Farr, Victor LaBolle, and Norman E. Willmorth. Planning guide for computer program development. Technical Report TM-2314/000/00, System Development Corporation, Santa Monica, California, May 1965.
- [26] Norman Fenton and Shari Lawrence Pfleeger. *Software metrics (2nd ed.): a rigorous and practical approach*. PWS Publishing Co., Boston, MA, USA, 1997.
- [27] Tom Gilb. *Software Metrics*. Winthrop Publishers, Cambridge, Mass, 1977.
- [28] Robert B. Grady and Deborah L. Caswell. *Software Metrics: Establishing a Company-Wide Program*. Prentice Hall PTR, 1987.
- [29] T. E. Hastings and A. S. M. Sajeev. A vector-based approach to software size measurement and effort estimation. *IEEE Trans. Softw. Eng.*, 27(4):337–350, 2001.

- [30] Douglas W. Hubbard. *How to Measure Anything: Finding the Value of "Intangibles" in Business*. John Wiley & Sons, Inc., 1 edition edition, August 2007.
- [31] Capers Jones. *Applied software measurement: assuring productivity and quality*. McGraw-Hill, Inc., New York, NY, USA, 1991.
- [32] T. C. Jones. Measuring programming quality and productivity. *IBM System Journal*, 17(1):115–125, 1978.
- [33] B. Kitchenham, S.L. Pfleeger, and N. Fenton. Towards a framework for software measurement validation. *Software Engineering, IEEE Transactions on*, 21(12):929–944, Dec 1995.
- [34] V. LaBolle. Statistical analysis of computer programming costs. In *SIGCPR '66: Proceedings of the fourth SIGCPR conference on Computer personnel research*, pages 29–38, New York, NY, USA, 1966. ACM.
- [35] M. J Lawrence. Programming methodology, organizational environment, and programming productivity. *Journal of Systems and Software*, 2(3):257–269, September 1981.
- [36] L. Buglione and A. Abran. Multidimensional software performance measurement models: A tetrahedron-based design. In R. Dumke/A. Abran, editor, *Software Measurement: Current Trends in Research and Practice*, number 3-8244-6876-X, pages 93–107. Deutscher Universitäts Verlag GmbH, 1999.
- [37] Alan MacCormack, Chris F. Kemerer, Michael Cusumano, and Bill Crandall. Trade-offs between productivity and quality in selecting software development practices. *IEEE Softw.*, 20(5):78–85, 2003.
- [38] Katrina D. Maxwell and Pekka Forselius. Benchmarking software-development productivity. *IEEE Softw.*, 17(1):80–88, 2000.
- [39] Katrina D. Maxwell, Luk Van Wassenhove, and Soumitra Dutta. Software development productivity of european space, military, and industrial applications. *IEEE Trans. Softw. Eng.*, 22(10):706–718, 1996.
- [40] Sandro Morasca and Giuliano Russo. An empirical study of software productivity. pages 317–322, 2001.
- [41] Simon Moser and Oscar Nierstrasz. The effect of object-oriented frameworks on developer productivity. *Computer*, 29(9):45–51, 1996.
- [42] John Moses, Malcolm Farrow, Norman Parrington, and Peter Smith. A productivity benchmarking case study using bayesian credible intervals. *Software Quality Control*, 14(1):37–52, 2006.
- [43] E.A. Nelson. Management handbook for the estimation of computer programming costs. Technical Report AD-A648750, System Development Corporation, October 1966.
- [44] D.J Reifer. Let the numbers do the talking. *CrossTalk The Journal of Defense Engineering*, pages 1–8, March 2002.
- [45] W. Scacchi. Understanding software productivity: a comparative empirical review. *System Sciences, 1989. Vol.II: Software Track, Proceedings of the Twenty-Second Annual Hawaii International Conference on*, 2:969–977 vol.2, Jan 1989.
- [46] Erik Stensrud and Ingunn Myrteit. Identifying high performance erp projects. *IEEE Trans. Softw. Eng.*, 29(5):398–416, 2003.
- [47] Charles R. Symons. *Software sizing and estimating: Mk II FPA (Function Point Analysis)*. John Wiley & Sons, Inc., New York, NY, USA, 1991.
- [48] C.R. Symons. Function point analysis: difficulties and improvements. *Software Engineering, IEEE Transactions on*, 14(1):2–11, January 1988.
- [49] III Vernon V. Chatman. Change-points: a proposal for software productivity measurement. *J. Syst. Softw.*, 31(1):71–91, 1995.
- [50] J. Vosburgh, B. Curtis, R. Wolverton, B. Albert, H. Malec, S. Hoben, and Y. Liu. Productivity factors and programming environments. In *ICSE '84: Proceedings of the 7th international conference on Software engineering*, pages 143–152, Piscataway, NJ, USA, 1984. IEEE Press.
- [51] C. E. Walston and C. P. Felix. A method of programming measurement and estimation. *IBM System Journal*, 16(1):54–65, 1977.